1 Predavanja

PREVAJALNIK

jezikovni, ko se poklicna tool C /paradigm/ (vmes OOP, funkcijski, logicni, telesnem)

knjizni pokazov

- ne bomo prodal o kvem vzaicnem

cilj: da znamo smetra = lexer C kompiler

1 teden = 1 predavanja ni "kompiler implementation in java"

optional "compilers principles, techniques and tools"

Vsebina ON	n - velikost = $\sum_{i=0}^{n} i$ = velikost $\frac{n^2}{2}$

ce vemo strategijo 8 predavanj na videnca = vseph je easy

kopit: spremeni vsoj compiler tekst da _

Vsaj 50% razumeš ON

50 vaje + 50 izpit

dobre predmet

Na vajah se nemi + vsepleda

Struktura prevajalnika

struktura je podobna cevovodu

vstop izvorne kode

izvorna koda → (1) leksikalna analiza → (2) sintaksna analiza → (3) abstraktni sintaksni drevo

(1) primer: Beri on 5 3 29; beseda

(2) vsevrsti ali bod in napaki dobro vsepleda

(3) poenostavi, odstrani vsepleda, ki ne rabimo za nadaljno

(3) → semantična analiza → (4) tabela simbolov → generiranje vmesne kode → (5)

(4) preveri, ce je vse pravilno definirano, ce se uporablja pravilno

(4) tabela simbolov

(5) vse kode je neodvisno od ciljne arhitekture

(5) generiranje vmesne kode → (6) kar pomeni lahko

(6) optimizacija vmesne kode → (7) generiranje ciljne kode → (8)

(6) optimizira, popravlja ali odstranjuje nepotrebne stvari v kodi

(7) odvisno od ciljne arhitekture

(8) dodeljevanje registrov → (9) izpis ciljne kode → ciljna koda

(8) Registrov je vedno manj kot spremenljivk, ki rabijo za izračun

(9) izpis v datoteko

1 faza = 1 teden (razen semantične analize)

od lekse on do gen vmesne kode = sprednje del

(od vmesnega gre do vmesne kode)

od vmesne kode do ciljne kode = zadnje del

GCC ABI

C / C++ / Fortran / ... → vmesna koda → x86 / ARM / RISC

LEKSIKALNA ANALIZA

izhod: program v izvornem jeziku

vhod: nizovni: Unicodeovih simbolov

izhodni: simbol

1. vrsta leksikalnega simbola (ime, kind, simbol)

primer: ime, variabla, besedna, 0, 1

nadalje: operatorski, oklepaj, vejica, podpicje, ...

ne kaze na kaj

ime funkcije

- ključna beseda
- operator
- oklepaj

} določeno s izvornim jezikom

2. vrednost podatkovni (lahko v simbolu)

3. položaj v izvorni datoteki (kje je napaka vrstica, odmik, ...)

razvijes za odkrivanje napak

ime tipa

INT, ID (ime), LP, RP, LCPAR, RCPAR, LB, INT LIT (10), ASSIGN, ...

ime vrsta

v oklep. je assoc. (4,0).

INT ∈ NST (10), SC, 10(4).

izhod leks. analize (tipedef)

samo primitivni tipi se uporabljajo, ostale se razpisne kot INT

ce je zamenjavi, kaj nadaljnjem programu ne pove, se potem "ukipa" tipa

(v nasem primeru)

programu je "prazno", ce se vsepe pretvori

for ← ključna beseda

for for 1(vrstica 3 ∈ pravilo prog

} v datoteko leks analizo

vsota ukipa tipa ce bo se kam prevzeto nikt nepo

a) leks leksikon ((;) = , ≠ ...) leks analizator izpravimo (v nasem primeru prog jezo)

b) komentarje

ime [A-za-z_][A-za-z_0-9]*

beseda for for

leks beseda [a-z]+

ena stevilka [0-9]+

hex st [0-9]+([0-9a-fA-F]+)

komentarji v pascalu

komentarji v C: /*([0-9A-Za-z_]|\n|\t|\r|" ")*/

avtomat za: ime

A-Z, a-z → 1 → 0-9, A-Z, a-z, _ → 1

1 → WS → NIKA

1 → 0-9 → 1 → INTCONST

FOR

DIKA

8 stanj

ID

2 načina razpoznavanja reklo Switch

	ASCII ZNAK
	v kakšnem stanju sprejmo

(Dolzje: ali je ime keyword)

1. leks analiza se je ustavila
- najdaljši leks. simbol, ki velja (longest match)

2 numerične konstante se razpoznavajo

123 → CONST (123)

+123 → PLUS, INTCONST (123)

-123 → MINUS, INT CONST (123)

Antlr → orodje za generacijo DKA

(demo izpis Regex, koda gp v learn)

DN

naloge vsote leks. simb

spisati regex za ...

2. Predavanja

Gbrez ov. kontekstno odv. gr. — T S — LCA linearna — avtomati

Sintaksna Analiza

KNG LG — SA LCA — lin gramatika

vhod: zaporedje leksemov / simbolov

izhod: drevo izpeljav (sled izpeljave)
zaporedje produkcij v izpeljavi

dve vrsti izpeljav:
- skrajna desna izp. (RM) — navzgor — skrajno desni vmesni
- skrajna leva izp. (LM) — navzgor — najdemo skrajno levi vmesni simbol

Sintaksno gezdra opisemo z KNG (ni nujno, je pa "najbolj" formalizem) primer: Java Language Specification 7. ed.

KI JE NEDVOUMNA

$$E \to (E) \mid M * E \mid E * M \mid M$$

$$M \to A \mid M + A \mid A + M \qquad x+y*z$$
$$A \to A+N \mid N+A \mid N$$
$$N \to 0 \ldots 9$$

algoritmi:
- LL: left to right scan producing the left most derivation
  - alg: navedimo levo sled izpeljave od zgoraj navzdol
- LR: L... producing the right most derivation in reverse order
- PEG: parsing expression grammars
- sintaksni kombinatorji (syntax combinators)

kanonični LL: LALL, SLL, ALL(*), GLL, ...
kanonični LR: LALR, SLR, ..., GLR, ...

LL je malo šibkejši od LR, je pa lažje za impl.

na vajah bomo uporabili LALR / ALL ? ne vem

LL ima boljše obveščevanje o napakah kot LR

LL omogoča "hacke" :D

LR in LL sta linearna v časovni kompl.

LL algoritem

primer: $S \to AB$, $A \to aA \mid b$, $B \to ab \mid ba$

predpostavka: vhod se razvije iz AB

$S \Rightarrow_{lm} AB \Rightarrow_{lm} aAB \Rightarrow aaAB \Rightarrow aabB \Rightarrow aabba$
$S \Rightarrow_{lm} AB \Rightarrow_{lm} bB \Rightarrow bab$

če pogledamo 1 simbol na produkcijo v katero "smer" gremo

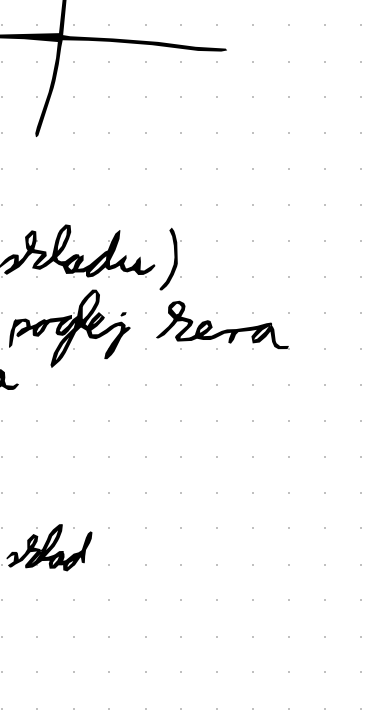
```
void parseS(){
    switch (peek()) {
        case 'a':
        case 'b':
            parseA()
            parseB()
            break
        default: error();
    }
}
```

```
void parseA(){
    switch (peek())
        case 'a': next('a'); ← 'a' consumed
            parseA();
            break;
        case 'b':
            next('b')
            break
        default: error(); }
```

```
void parseB(){
    switch (peek())
        case 'a':
            next('a');
            next('b');
            break
        case 'b':
            next('b');
            next('a');
            break
        default: error();
    }
}
```

rekurzivno spuščanje

primer: vhod aaabba $ ← EOP



vhod	sklad	vzhod
aaabba$	S$	S → AB
aaabba$	AB$	A → aA
aaabba$	aAB$	
aabba$	AB$	A → aA
aabba$	aAB$	
abba$	AB$	A → b
abba$	bB$	
ba$	B$	B → ba
ba$	ba$	
a$	a$	
$	$	

leva sled izpeljave

	a	b	$
S	S → AB	S → AB	
A	A → aA	A → b	
B	B → ab	B → ba	

if (vrnemo simbol na skladu)
  pogledj "top" - kdo, pogledj kera produkcija jo da

else
  if ob vhod == sklad
    izbriši
  else
    error

S → AB, A → Aa | b, B → ba — bab ba

	a	b	$
S		S → AB	
A		A → b A → Aa	
B		B → ba	

(4) gramatika ima tabelo celic kjer ima samo 1 produkcijo na celico

kdaj enakost pogledamo na začetku

S → ab | ac ∉ LL(1), ∈ LL(2)

E → E + T | E − T | T
T → T * F | T / F | F
F → id

- ni tipa LL(1), je LR(1)
- se ne da napisati v LL, ker imamo levo rekurzijo in imamo zpet več produkcij v isti tab

E - je naredila pol jubo
T ger

Ē → TE'
Ē' → ε | +TE' | −TE'
T → FT'
T' → ε | *FT' | /FT'
F → id

tipa LL(1)

ANTLR podpira oboje, samo ugor nja je počasnejša

↳ boljše, ker je linearno

3. Predavanje

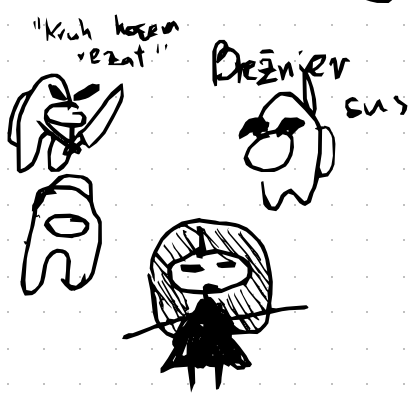
Abstraktno sintaksno drevo

↑

drevo izpeljave

"Zakaj hodite na faks? Da vjer ss si druge zobje polomil, da si naute tied vi."

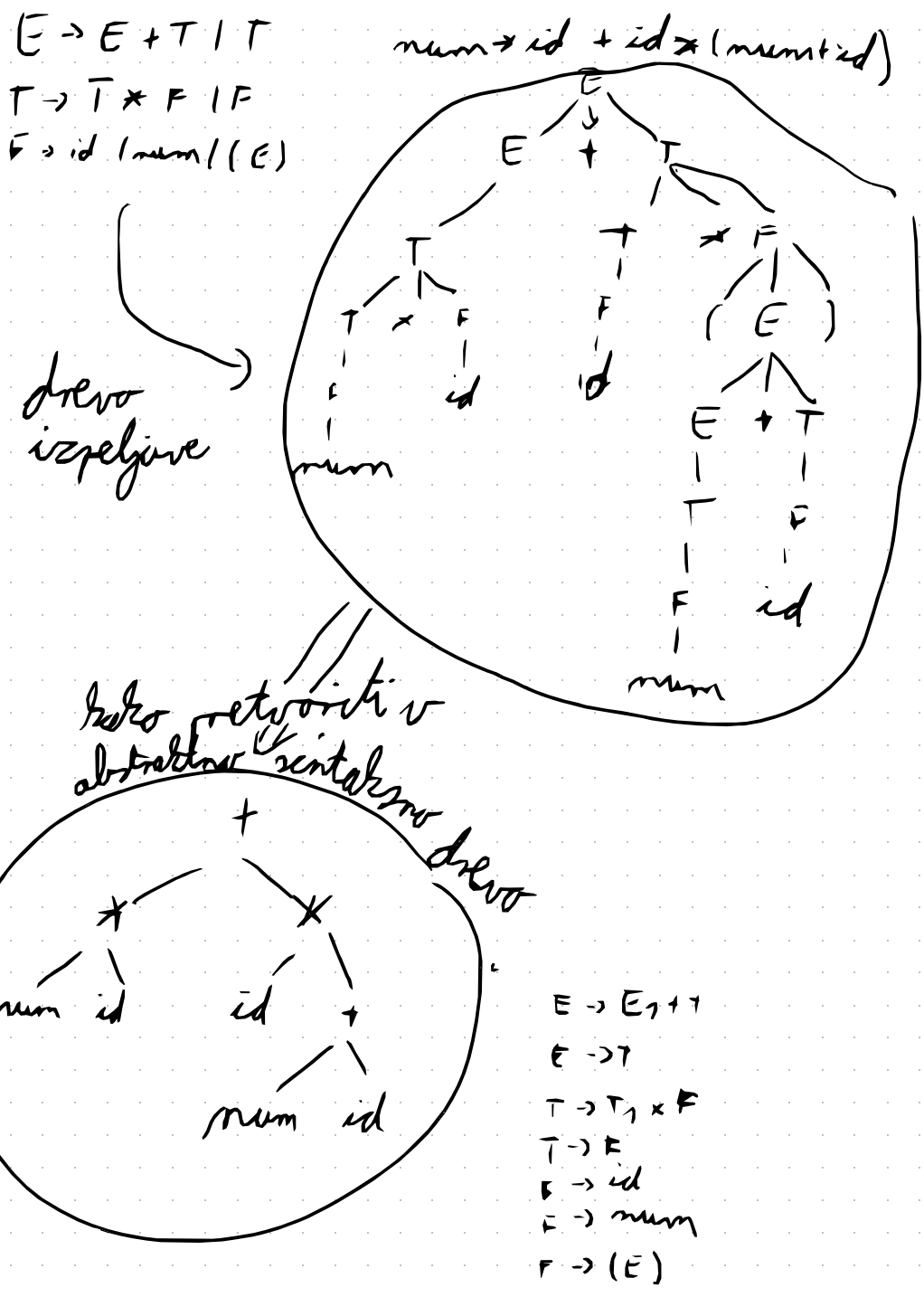
– Slivnik 2025 –



ABSTRAKTNA SINTAKSA

VHOD: drevo izpeljav

IZHOD: abstraktno sintaksno drevo
(in skrt, nova gramatika poenostavlja drevo izpeljave)



E.AST = +(E1.AST, T.AST)

E.AST = T.AST

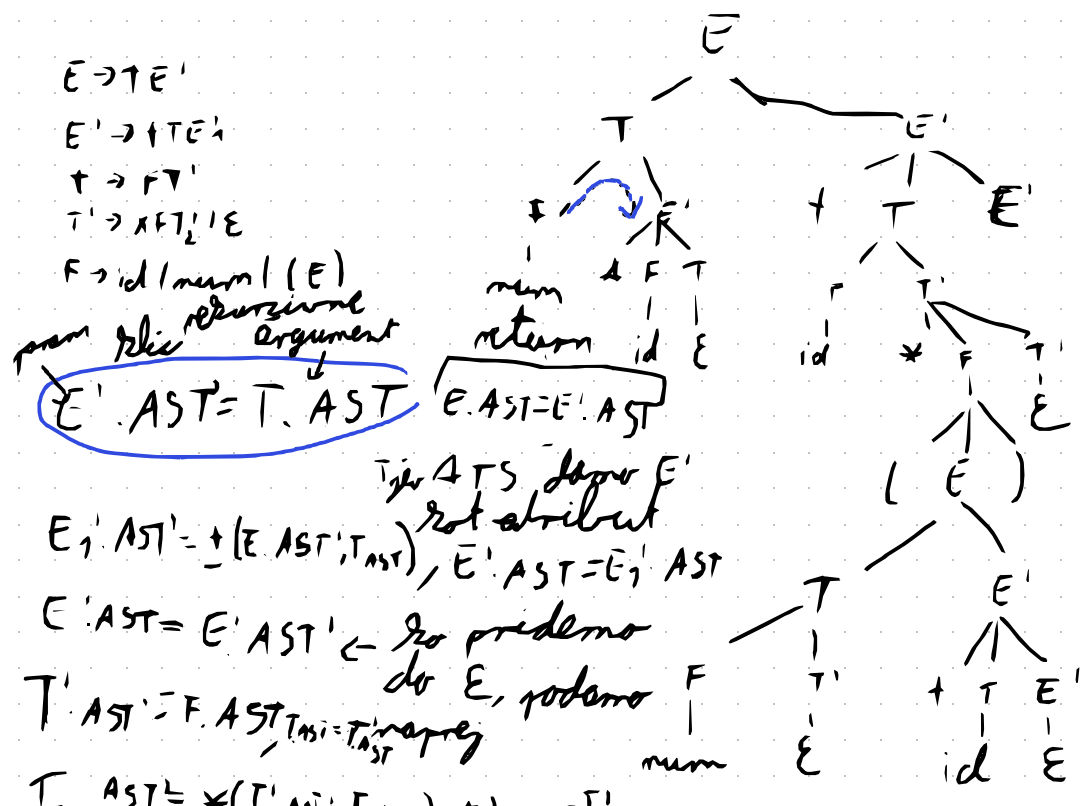
T.AST = F.AST

T.AST = *(T1.AST, F.AST)

F.AST = num

F.AST = id

F.AST = E.AST




```
ast parseE(){
    ast t = parseT();
    return parseE'(t);
}

ast parseE'(ast tLevi){
    switch(token.peek()){
        case '+': skip(+);
                  ast tDesni = parseT();
                  return parseE'(+(tLevi, tDesni));

        case 'ε':
                  return tLevi;
    }
}
```

vrednost atributa na levi odvisna od vrednosti atributa na desni

← smer računanja

AST = podedovan atribut

$A \to X_1 X_2 \dots X_n$

$attr(X_{k+1}) = f(attr(A), attr(X_1), \dots, attr(X_{k-1}))$

Sintezno usmerjeno prevajanje

od izvorne kode do abstr. sint. drevesa

in nato prevajanje poddreves

orodje: atributne gramatike

((KNG + atributi + semantična pravila za izr. atributov

LR bolj močen od LL, LL nudi boljši opis napak kot LR

4. Predavanja | Semantična analiza

vhod: abstraktno sintaksno drevo
izhod: abstraktno sintaksno drevo + atributi
↑
ki računamo za odvisno od prog. jezika

Kaj se dogaja
- razreševanje imen:
  - ugotovimo za vsako ime: kje je def / uporabljeno
    vsako uporabljeno mora biti nekje def

- preverjanje tipov: ali so vsi klici opravljeni v pravilnem sistemu tipov
- odstranjevanje nedosegljive kode (nerabljeno, zato lahko zmanj povedu, a je ker nedosegljivo)
- analiza strogosti v kodi (strictness računanje)
  nonstrict - je
  strict -

Kaj lahko delamo:
- statično (pred izvajanjem) - strožje, hitrejše izv.
- dinamično (med izvajanjem) - bolj prosčena pravila, počasneje

Zakaj statično strožje ⇒ kot din?
Ker je problem izračunljivosti

semantična analiza ignorira if

```
int x = 1;        x = 1
x = x + 1;        x = x + 1
```
| x : int

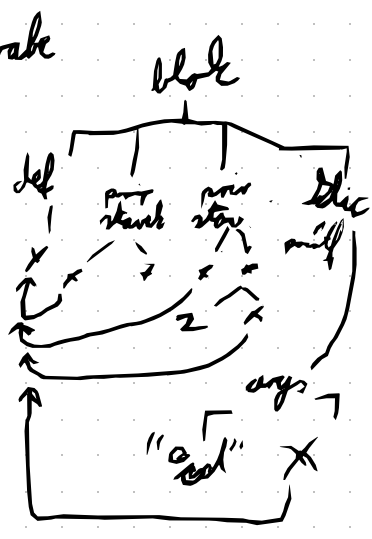
```
if() x = "5";     if( )x = "5";
x[0]              x[1]
```
| x : string

Razreševanje imen:

- ali so vsa imena "znana, definirana, ..."
- povezava mesto definicije in mesto uporabe

primer: C

```
{ int x;          ← mesto definicije
  x = 5;
  printf("%d", x);   ← mesto uporabe
  x = 2 * x;
}
```



problem
- imenski prostor (namespace)
- doseg (scope)

Senčenje

```
int x {
  int f (int y) {
    return x + y;
  }
  int g (int x) {
    return x + x;
```

1

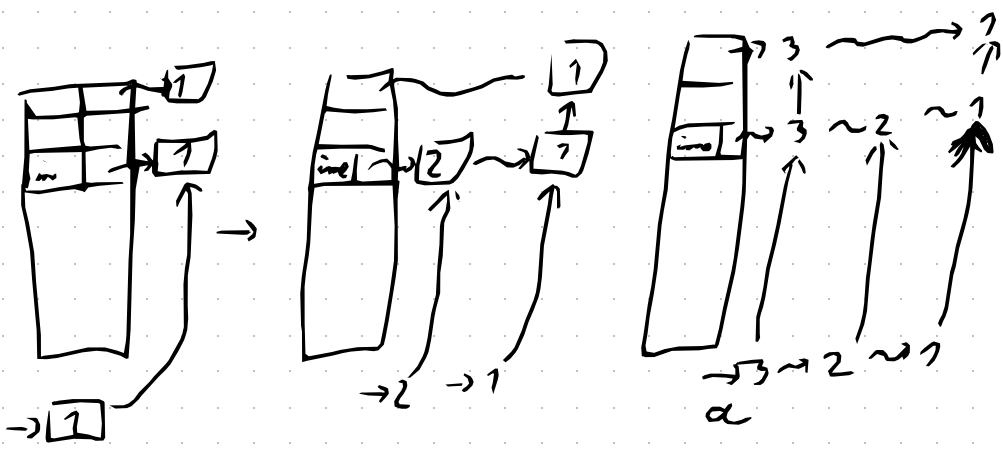
Razreševanje imen:

simbolna tabela
  preslika imena v definicijo

med obhodom AST
- na mestu definicije imena
  vstavimo (ime, def) v sim. tab.
  če je ime v tabeli ⇒ javimo napako
  ⇒ nova def zasenči staro

- na mestu uporabe imena
  če def obstaja: usmerimo kazalec na def
  če def ne obstaja: javimo napako

O(1) ↗
ins: doda ime, def v simtab ali javi napako
O(n) find: vrne def danega imena ali javi napako
O(1) new_scope: začne novo področje dosega
O(n) old_scope: konča trenutno področje dosega



če želimo izbrisati scope, začnemo pri a, zbrišemo vse do naslednjih linka, potem pa sebe zbrišemo / poppa

5 Predavanja nazaj Semantična analiza

za

Tip je množica vrednosti

(cela števila, plavajoča števila, ...)

nekdo ne pravi, da tip ne more biti število × {T, F}

algebrajski tipi (primer v C-ju - struct/union)

Tip nam pove s katerimi se lahko dela in operatorji, ki jih lahko uporabimo

tipizacija

statična
- v času prevajanja
- lahko povemo tip za poljubno spremenljivko

dinamična
- v času izvajanja
- spremenljivke nimajo tipa

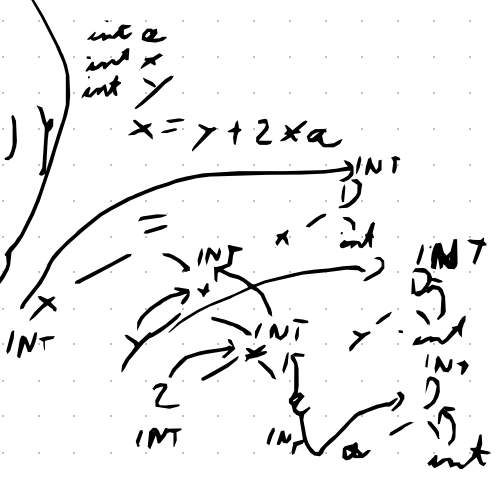
JAVA:
class A, {a()}
class A' extends A {a'()}

A x = new A'()

nekaterih lahko vidi samo a()

int a
int x
int y

x = y + 2 * a



Kaj zdej moramo pri statičnem preverjanju

1. za vsak tip v prog določimo opis IN predstavitev
   - vgrajeni tip (int, char, ..)
   - deklarirani (struct, union, class, ..)
2. preverjanje tipov
   - ali so pri vseh operacijah pravilni tipi

rekurzivni podatkovni tipi

C: struct node {
    struct node *p1, *p2, *p3;
};

struct A{struct B *b;} v javi
struct B{struct A *a;}

class A{B b;}
class B{A a;}

---

tip A = <b : ^B>
tip B = [a : A]

Prevajanje levih vrednosti (L-values)

leva vrednost = vrednost, ki ima naslov

1 //
1+1 //
x ✓
(int)1 ✓
p ✓ p je kazalec
*p ✓

zakaj pa? ker pri preverjanju tipov mamo lahko

z = 3 * 5
↑
vendar tega ne prepisevamo ker ni leva vrednost.

Izračun konstantnih podizrazov (boxed type?)

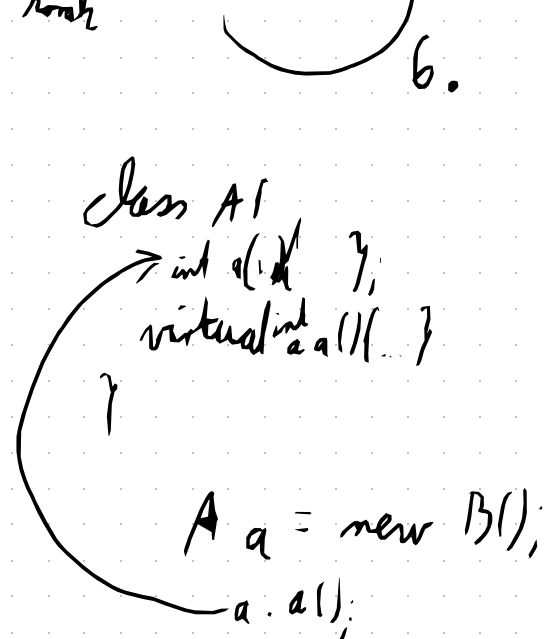
- ker smo ravno tukaj
- ker včasih rabimo

prebrežene metode

int max(int i, int j){ }
int max(double a, double b){ }
int max(int i){ ... };

x = max(2, 3)
x = max(x1, x2) ← katero funkcijo kliče? odvisno od tipov x1, x2

typedef struct {int z;} Z;



class A{
  int a(){ }
  virtual int aa(){ }
}

class B{
  int a(){ }
  virtual int aa(){ }
}

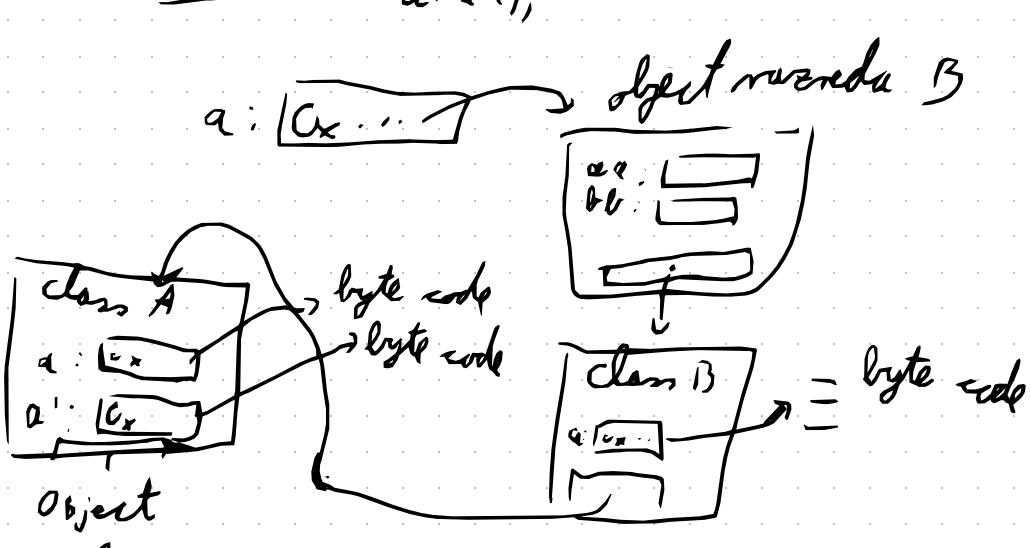
A a = new B();
a.a();
a.aa();

---

class A{
  int a;
  int a(){ }
  int aa(){ }
}
class B extends A{
  int b;
  int a(){ }
}

SLIVKO MI DAJE DR HOUSE VIBES

A a = new B();
a.a();
a.a'();



če želimo preveriti, ali je a instanceof A, bo šel v a, sledil v object razr B, pol v class A.

in short, gre dokler ne najde ali pa pride do razreda Object

6. Predavanje

KLICNI ZAPISI IN SPREMENLJIVKE

vhod: AST z atributi
izhod: AST z še več atributi

spremenljivke:
- statične } to bomo delali
- avtomatske
- registerske
- cevnanje

statične:
- fiksni naslov
- obstajajo ves čas

avtomatske:
- spremenj. naslov
- čas izvajanja funk

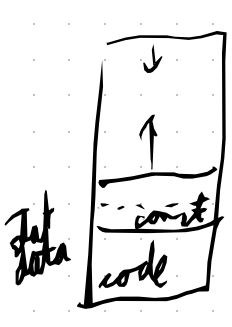
registerske:
- spremenljivke podane v register

cevnanje:
- ista imena
- fiksni naslov

Kaj rabimo pri:

statični:
- ime ← ime labele
- velikost
- vse ost.

avtomatska:
- odmik (od kazalca stack)
- velikost



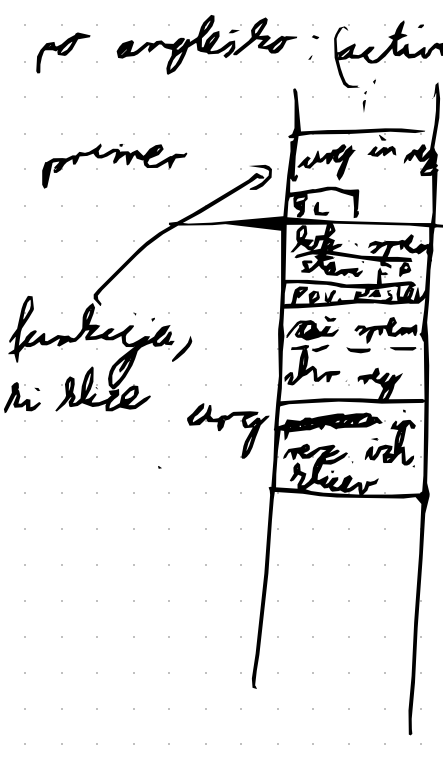
← stat. data se da neč ~1/2, ž so konstantne in velikost

komponente zapisov
- vrstny spremenljivke, odmik, velikost
↑ glede na sklad v funkciji

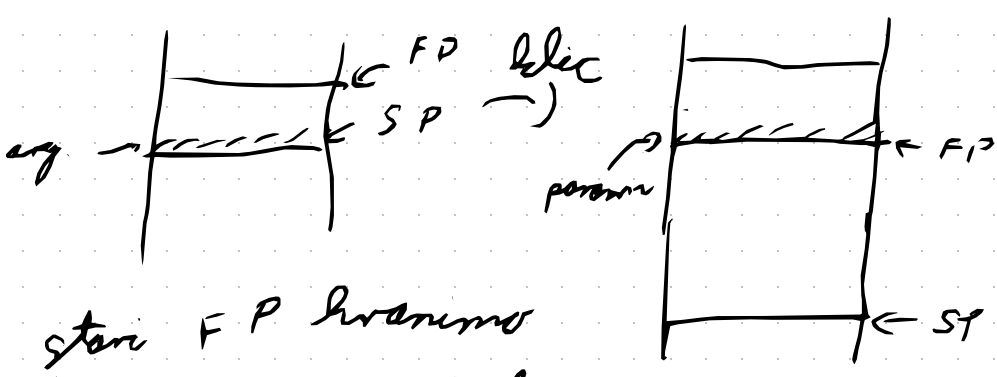
---

klicni zapis - del sklada, ki pripada funkciji med izvajanjem

po angleško: (activation record) ali stack/call frame



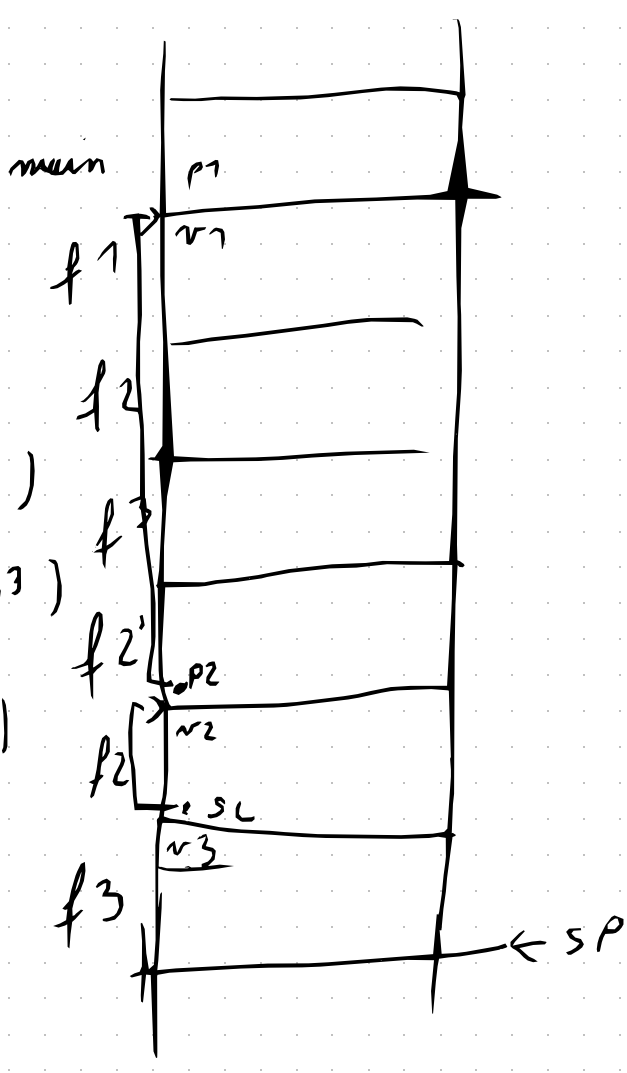
SL - statični kazalec



stari FP shranimo da ne memory leaka

začasne & shran. reg → registri, ki se shranijo pri klicu

spremenljivke, ki jih hočemo imeti v registrih, ampak jih ni




```
fun f1(p1: )
  var v1
  fun f2(p2: )
    var v2
    fun f3(p3: )
      var v3
  fun f2'(p2': )
    var v2'
```

SL → kaže do argumentov funkcij, ki v klicalki trenutno funkcije

primer: (v funkciji f3) dostop do

v3	LD $1, [FP-4]
p3	LD $1, [FP+8]
	↑ size of void* na 64 bit
	registri ↓
SL	LD $0, [FP+0]
v2	LD $1, [$0-4]
p2	LD $1, [$0+8]
SL	LD $0, [FP+0]
SL→SL	LD $0, [$0+0]
v1	LD $1, [$0-4]
p1	LD $1, [$0+8]

SL se pojavi samo v gnezdenih funkcijah

DL - dynamic link se bi uporabil heap, zato se veže na en stack frame pred tem

7. Predavanje

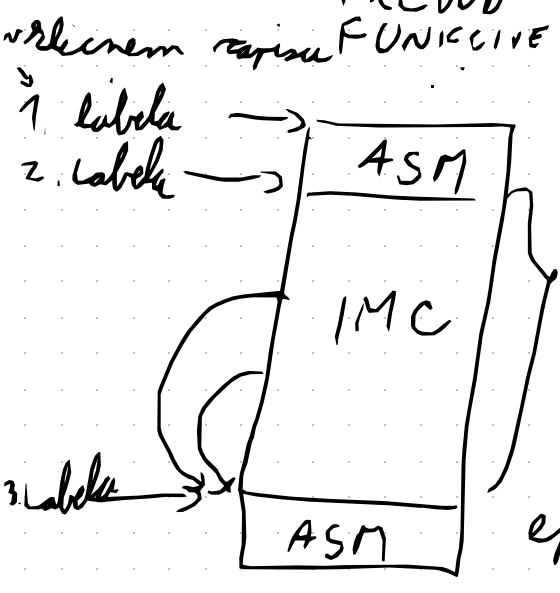
Generiranje vmesne kode

VHOD: AST + atributi

IZHOD: vmesna koda za posamezne funkcije

PREVOD

vrhovnem razredu FUNKCIJE



ASM - assembler

3-addressna vmesna koda

MOVE $1, $2

ADD A, B

+ addressna vmesna koda

ADD $1, $2, $3 => $1 = $2 + $3

drevesna vmesna koda



skladovna vmesna koda

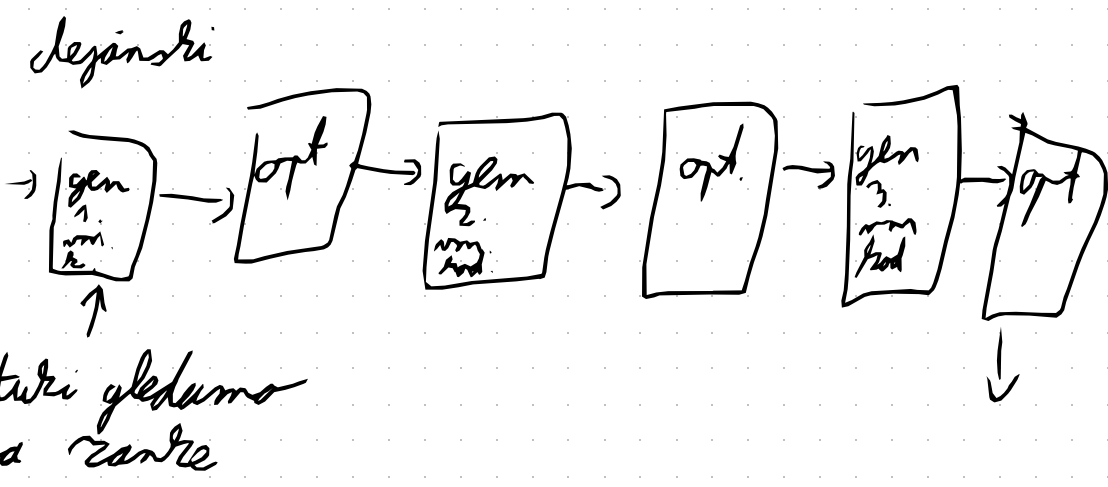
java byte code

---

enostavni prevajalniki

→ GEN VMES. → slika vmesne kode

dejanski



tukaj gledamo za razne

vrste

CONST

NAME

TEMP

BINOP

12 => CONST(12)

12 * 3 + 4 → BINOP(+, BINOP(*, CONST(12), CONST(3)), CONST(4))

globalna spremenljivka X → MEM(NAME(_X))

lokalna spremenljivka X odmik -16 na tem nivoju → MEM(BINOP(+, TEMP(FP), CONST(-16)))

na nivoju dalje



MEM(BINOP(+, MEM(n), CONST(-16)))

FP zunanje funkcije

TEMP(FP)

komponenta zapisa

X.c → odmik +8 → MEM(BINOP(+, BINOP(+, TEMP(FP), CONST(-64)), CONST(8)))

lok. spr. na tem nivoju = 64

element tabele

X[17] → MEM(BINOP(+, BINOP(+, MEM(MEM(TEMP(FP))), CONST(-512)), BINOP(*, CONST(17), MEM(NAME(-Y)))))

lokalna spremen. 2 nivoja ven, odmik -512

vzorci za stavke

JUMP
MOVE
LABEL
CJUMP

x = 2 * y

lokalna spr. -8 ; glob

MOVE(MEM(BINOP(+, TEMP(FP), CONST(-8))), BINOP(*, CONST(2), NAME(_Y)))

ko je MEM destination, je to store

use ostalo je load

if x==3 then x=3 → y

CJUMP(BINOP(==, MEM(BINOP(+, TEMP(FP), CONST(-8))), CONST(3)), L1, L2)

to je load

then

LABEL(L1) JUMP(L2) | LABEL(L2)

exit

8 predavanja

linearizacija vmesne kode
vhod: vmesna koda poamezne funkcije
izhod: linearna vmesna koda
1. grezdem vrhi funkcij

1. grezdeni vrhi funkcij

f(f2(f3(0)))

1. SL za f1 na sladu
2. SL za f2 na sladu

2. vhod SEXPR

3. vhod CJUMP

problem: 2 rezultata sta podana, izhodi kam
da nimamo dovolj prostora v stogu
rešitev za sta

Branch T, branchF

cilj:

MOVE
TEMP CALL

za non-void funkcijo

ali

ESTMT
CALL

ta je za calleraj void fun.

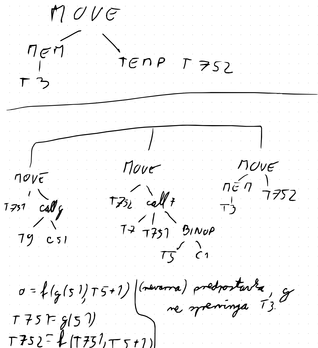
CALL je odvisen od procesorja, zato ga
želimo čim bolje odstraniti

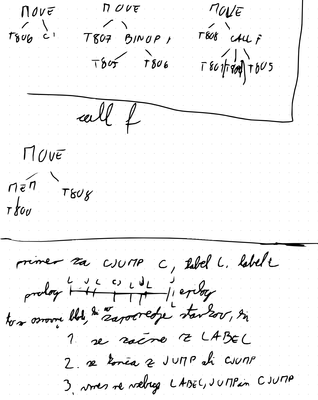
KAKO LINEARIZIRAMO

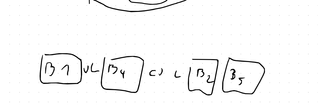
rekurzivno po drevesu in rekurzivno

dve vrsti vmesne kode:
- stavke stavek -> (seznam stavkov)
- izraze izraz -> (seznam stavkov) izraz

primer







a = f1(g(S1), T5+1)
T751 = g(S1)
T752 = f1(T751, T5+1)
a := T752

(navarna) predpostavka, ga ne spreminja T3

call g podrevo
(tista kar smo spremenili)

call f

---

primer za CJUMP C, label L, label L

prolog L JC C, L, L epilog

kar smo imeli bbl je zaporedje stavkov, ki

1. se začne z LABEL
2. se konča z JUMP ali CJUMP
3. noben se nahaja LABEL, JUMP ali CJUMP

9. Predavanje

CJUMP C, L_T, L_F L_T

condition

if true LABEL to jump to

if false

L_T

CJUMP C, L_T, L_F', LABEL L_F', JUMP L_F



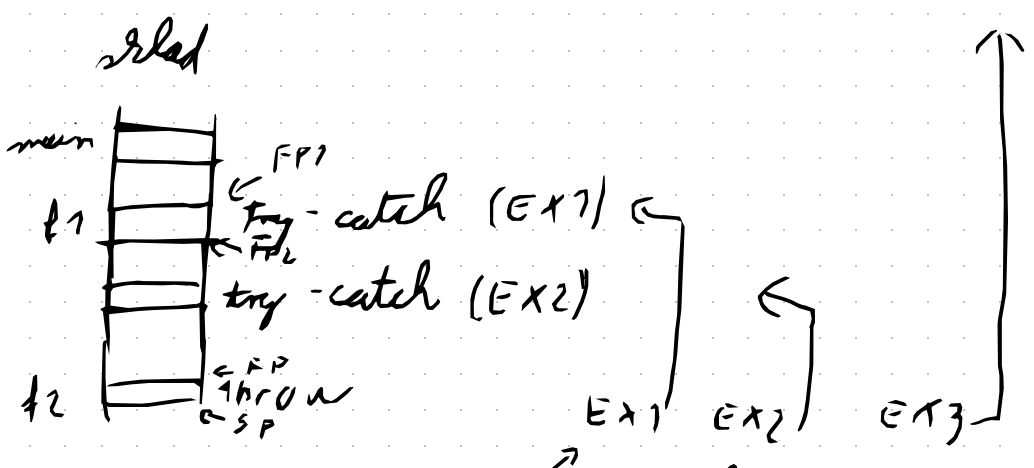
I/O v interpreterju:

& vidimo call jump _print, nikamor ne skačemo in to direktno izpišemo

---

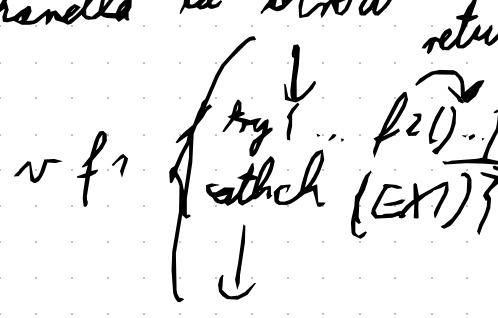
Izjeme:

C setjump ~ try - catch
longjump ~ throw



skačemo 1 kliceni zapis at a time in gledamo, če v tej funkciji lahko handlamo

ko se zgodi throw se trenutni FP in SP prepiše od tiste, ki lahko handla ta throw



pri try - catch se shrani PC counter, ki pridje do catch(..) stavka

throw pa dejansko restora context

setjump ⇒ naredi pointer na trenutni kontekst

funkcija

longjump ⇒ rekonstrukta kontekst preko pointerja

vedjamo dokler je funkcija živa

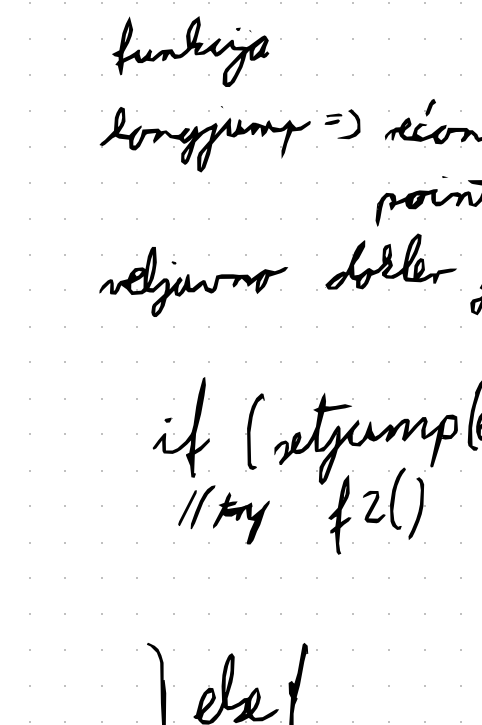
```
if (setjump(ex1)) {
  // try f2()
} else {
  // catch
}
```

longjump (context, err)

Avtomatsko čiščenje pomnilnika

2 metodi:
- preštevanje referenc (reference counting)
- označi in pomeči (mark and sweep)

1 int *p = NULL;
2 p = malloc(4);
3 int *k = *p; pri štetju referenc
4 p = NULL;



označi in pomeči: (shrani vse pomn.)
1. poišče vse kazalce med statičnimi spremenl., na skladu, ...
ko zmanjka spomina
2. vse kar je dosegljivo preko kazalcev in stat. spremenl. in na skladu
3. kar ni označeno: pobriši

problem: popolna ustavitev programa

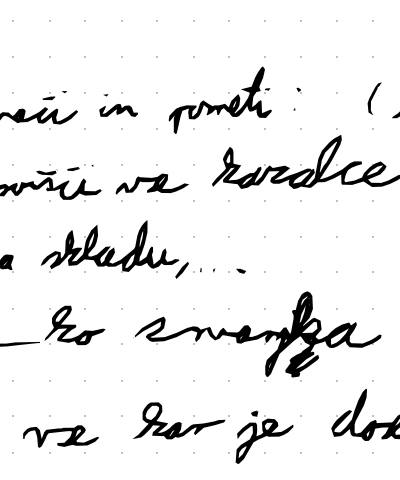
arena:



malloc, vsi grejo v arena 1, ko se zapolni do arene 2, se skočeja vse, in se se pravilno defragmentirajo

Generacije

stvari se po navadi hitro brišejo, če pa se ne, pa ostanejo



generacij se NE spreminja, razen ko ni več prostora vse razveljavi in poskusi skrčiti vse generacije

Kdaj stisne v generacijo? Ko doseže nekega thresholda, se spomin skrči in ustvari nova generacija

10. Predavanje

Generiranje strojnih ukazov

VHOD: (za posamezno funkcijo) zaporedje trenutnih drevesa (za CJUMP je false labela, ne levem delu MOVE je TEMP)

izhod: (za isto funkcijo) zaporedje strojnih ukazov za neki spomin

primer za drugo

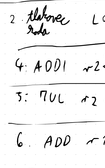


← shranjuje segment

uredimo vozne (shranjevanje / klicanje), da bojo pobirale te vrh segmenta povezalo se dejanska koda



← ima 2 outputa, 0 inputa



← ima 2 outputa, 1 input

1 shranjuje, prav, da je rezultat v spominu FP

2. shranjuje LOAD $r_1 \leftarrow M[FP+a]$

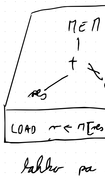
4. ADDI $r_2 \leftarrow r_{0,4}$

5. MUL $r_2 \leftarrow r_6 \times r_2$

6. ADD $r_1 \leftarrow r_1 + r_2$

8. LOAD $r_2 \leftarrow M[FP+X]$

9. STORE $[r_1+0] \leftarrow r_2$





Kakšen pa smemo drugačne vozne



2. LOAD $r_1 \leftarrow M[FP+a]$

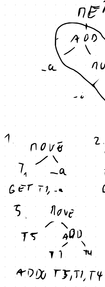
4. ADDI $r_2 \leftarrow r_{0,4}$

5. MUL $r_2 \leftarrow r_1 \times r_2$

6. ADD $r_1 \leftarrow r_1 + r_2$

8. ADDI $r_2 \leftarrow FP+X$

9. MOVEM $M[r_2] \leftarrow M[r_1]$



LABEL

OPER

└ MOVE

vsi ukazi razen ukaza za prepis vrednosti se enega v drugi register

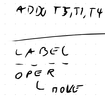
ADDI T1,T6,7

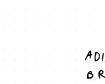
BR T1,L17 OPER

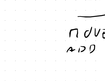
MOVE T1,T2

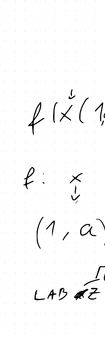
ADD T1,T2,0 ← ker je actually $V T1 = T2$, remote

$X\ r1 = r1 + x$, kjer $x \neq 0$







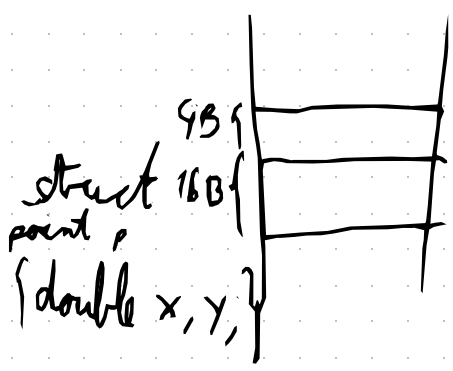


11. Predavanja

calling convention

primer: prvi 3 arg gredo v reg, ostali preko sklada

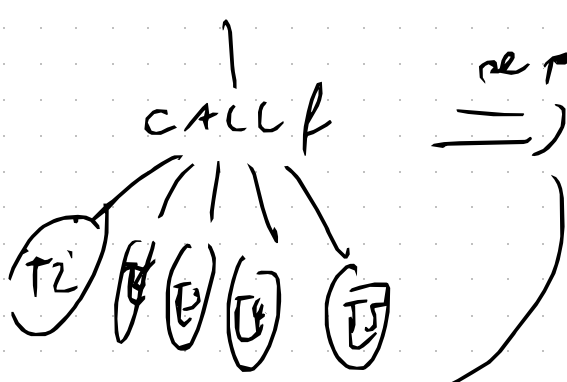
```
int f ( int a, int b, struct point p, int c, int d ) {
          R      R                  S      R     S
          $0     $1                 +0     $2    +16
```



(to je C, ninva SL)

če gl. klice f, je to klicni zapis

f



se preslika v zaporedje $\Rightarrow$ stopnih ukazov

```
code(T1) = res T1
code(T2) = res T2
code(T3) = res T3
code(T4) = res T4
code(T5) = res T5

LDO T6,[T3+0]
STO T6,[SP+0]
LDO T6,[T3+8]
STO T6,[T3+8]      } premik sklada

STO T5,[SP+16]

LD $0,T1
LD $1,T2           } a, b, c so zdaj v reg.
LD $2,T4

CALL f
STO $0,T0
```

Dodamo kodo, da shrani reg

MEM

T0

CALL f

OBNOVIMO rezultat v $0, če lahko, drugače po sklada

Aktivnost spremenljivke
hrani naslednjo stopnjo uporabe te spremenljivke

izhod — 1 1- in interferenčni graf

```
c := a + b
a := 0
if a > 0 goto L
b := a + 1
L: c := c + b
a := 2 * b
return c
```

→ vsaka spr. je aktivna med svojo prvo in zadnjo uporabo

Kaj to pomeni?
Za kateri pripravnik na 1 spremenljivko



use(n) = množica vseh spremenljivk, ki jih uporabe stavek n
def(n) = množica vseh spremenljivk, v katere stavek piše

naslednik
pred(n) = množica vseh možnih naslednikov stavka n
succ(n) = množica vseh možnih predhodnikov stavka n

in(n) = množica vseh spremenljivk, katerih vrednosti se morda uporabijo, še preden se stavek n izvrši
out(n) = množica vseh spremenljivk, katerih vrednosti se morda uporabijo v naslednjih korakih

```
q := 3        in() = {}
b := 5        in() = {}
c := a + b    in() = {a,b}
```

$in(n) = use(n) \cup (out(n) \setminus def(n))$

use "bere"
def "piše"

out je unija vseh in

```
1: a := 0
2: if a > 0 goto 4
3: b := a + 1
4: c := c + b
5: a := 2 * b
6: goto 2
7: return c
```

a je aktivna
b je aktivna
c je aktivna

Kako to izračunati?

	use	def	1. korak: in	1. korak: out	2. korak: in	2. korak: out
6	c	/	c	c	c	c
5	a	a	a, c	b, c	b, c	b, c
4	b, c	c	b, c	c	b, c	c
3	a	b	a	b, c	a	b, c
2	a	/	a, b, c	a, b, c	a, b, c	a, b, c
1	c	b	a, c	a, c	a, c	a, c
7	c	/	c	/	c	/

1. korak

	out	in
6	a, c	a, c
5	a, c	b, c
4	b, c	a, b, c
3	b, c	a, c
2	a, b, c	a, b, c
1	a, c	c
7	/	/

$in_0(n) = \emptyset$, $out_0(n) = \emptyset$

$in_{k+1}(n) = use(n) \cup (out_k(n) \setminus def(n))$

$out_{k+1}(n) = \bigcup_{m \in succ(n)} in_k(m)$

Ko je $in_{k+1}(n) = in_k(n)$ za vsak n, je algoritem končal.

Ja:
Algoritem se je ustavil pri $k = 1 + \max_n |in_k(n)|$
Kaj vsebuje $in_k(n)$? Spremenljivke, katerih vrednosti se lahko TEMU, kar se zgodi kot so uporabljene na poti, dolge največ k korakov.

Ne vsebuje $in_k(n)$ spremenljivke, ki se ne uporabi na poti, dolgi največ k korakov.

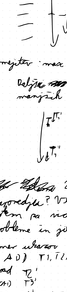
Algoritem se ustavi, ko je $in_{k+1}(n) = in_k(n)$ za vsak n.

Ja:
Algoritem se je ustavil pri $k = 1 + \max_n |in_k(n)|$
Kaj vsebuje $in_k(n)$? Spremenljivke, katerih vrednosti se lahko TEMU, kar se zgodi kot so uporabljene na poti, dolge največ k korakov.
Ne vsebuje $in_k(n)$ spremenljivke, ki se ne uporabi na poti, dolgi največ k korakov.
Algoritem se ustavi, ko je $in_{k+1}(n) = in_k(n)$ za vsak n.

a, b, c — interferenčni graf (povezava, če morajo biti v istem registru)

če lahko kopiramo v same b barvamo, lahko uporabimo isti register

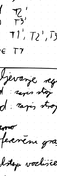
primer predavanj



Ta register potrebuje lahko dano vse v sp, če ne pa imamo problema?
Kaj naredimo? Spraženemo nekatere shrani kakor? Za vsako TEMP, ki se more kdaj v registrih nastopiti, naredimo na več kraje vzpomnilnik

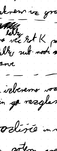
TEMPi

primer:



potrebujemo 3 registre

Poljuben T1 nastanejo na več mestih



Zakaj dobimo v takem razporedu? Vse dodelimo v registre, potem so vsi vmesni deli nam lahko problemi in jih dano na sklad

primer shranjevanja

```
ADD T1, T2, T3
LOAD T1
LOAD T3
ADD T1, T2', T3'
STORE T1
```

Razbarvanje registrov
Kako smo kdaj shranili interferenčni graf
izhod: naj bo kdaj shranili v registre

vsaj
interferenčni graf + K registrov

1. če kdaj vozlišče z $< K$ sosedi, ga umaknemo ve graf in na sklad
   - preostanku lahko pre
   - če ga ne moremo, gre kdaj
2. odstranimo vozlišče in ga vstavimo na sklad in ga razglasimo za vmesni pomen (spill)

(vozlišča in njene povezave ostranimo in potem gremo nazaj v 1 korak, potrebno shranjevanje grafa)

a dobimo DEJANSKI PREVIJ, to vozlišče damo na kakšno mapo

1, 2 => postavimo na vozlišča vs grafa na sklad
(odstranimo vozlišča iz grafa)
(odstranimo vozlišča, ki nimajo več povezav)

3. s skladom vozlišče = graf
   a) vozlišče lahko - $< K$ sosedov, ga pobarvamo
   b) ne: iščemo barvo in dodamo dejanski pomen

4. če: kakšno dejansko pomen razporedov kdaj interferenčni graf

drugače vzgoj

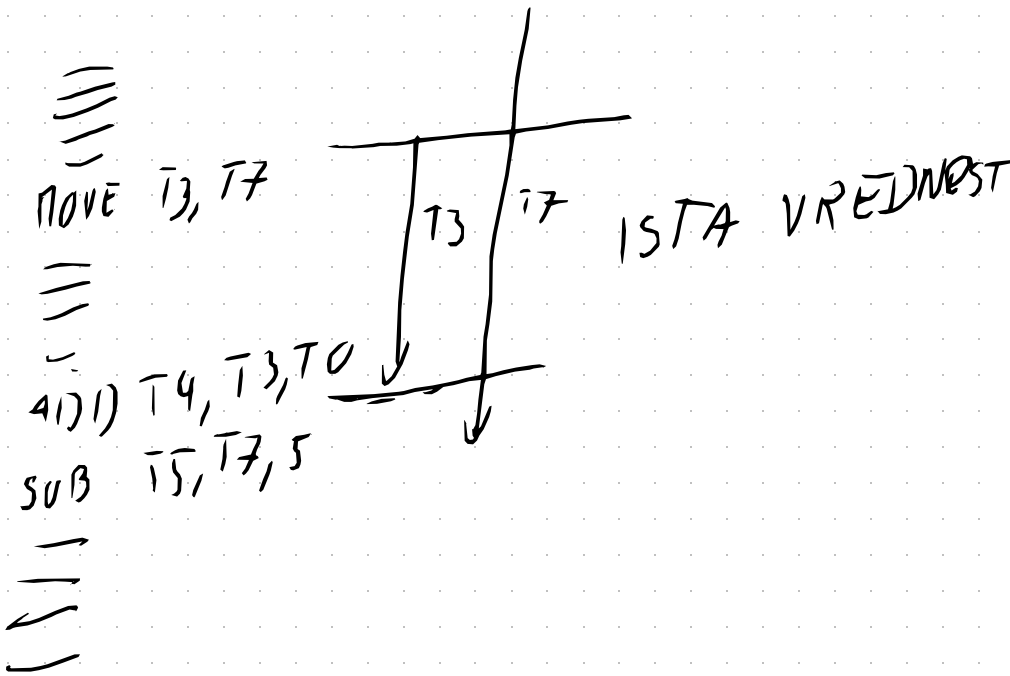
ideja: vozlišča sklanosti → razvrsti →

razporedi
shrani
(pomen)

13 Predavanje

Kaj smo delali?
Kako sparati calling convention
v registre

~~vijaček~~ move je funkcija



KI NISTA POVEZANI

če imamo 2 vozlišči, kateri seštevek je manj kot število barv (K), ju lahko damo skupaj

ZAKAJ? združevanje teh je $O(n^2)$, kjer je n št. vozlišč, proti barvanju ki je $O(c^n)$

ZAKAJ NE? graf postane bolj gost, ne, predvsem dovolj na performance

14. Predavanje

FP + SP registra se barvamo v vr. "11"

Napisati epilog in prolog

poglavje "putting it all together"

1. podatkovne definicije ← statične spremenljivke, velikost kratkih zbirk
2. programski deli

↓

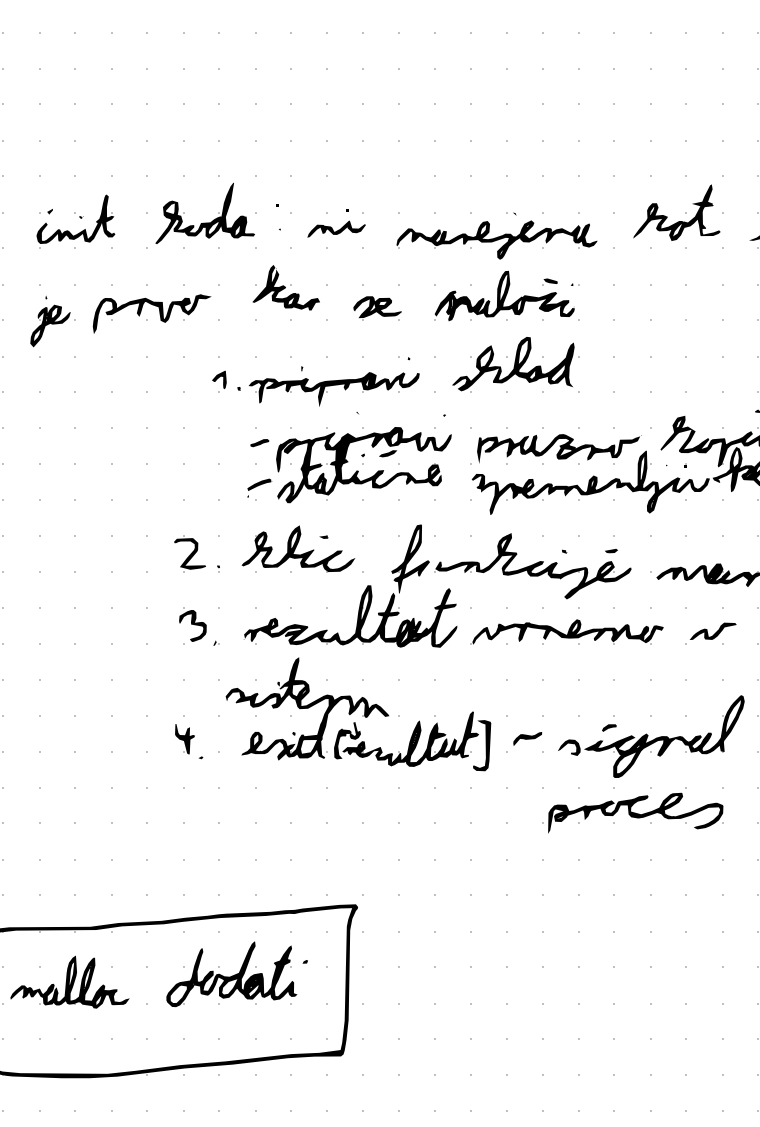
trenutno kot seznam funkcij

vsaka funkcija ima:
1. klicni zapis ← to je vsi MEM Frame
2. stanje izvajanja jedra

} ⇒ spravimo v zbirko

v zbirko

funkcija



ima predpono prolog

epilog

ima predpono

	Kaj se naredi
prolog	- shrani registre - ustvari klicni zapis - skoči na Lentry
epilog	- restavriraj registre - podremo klicni zapis - pripravimo rezultat - vrnemo RETURN (jump)

} kreacija je prva je odvisna od arhitekture

init koda ni nastavna kot funkcija in je prva kar se naloži

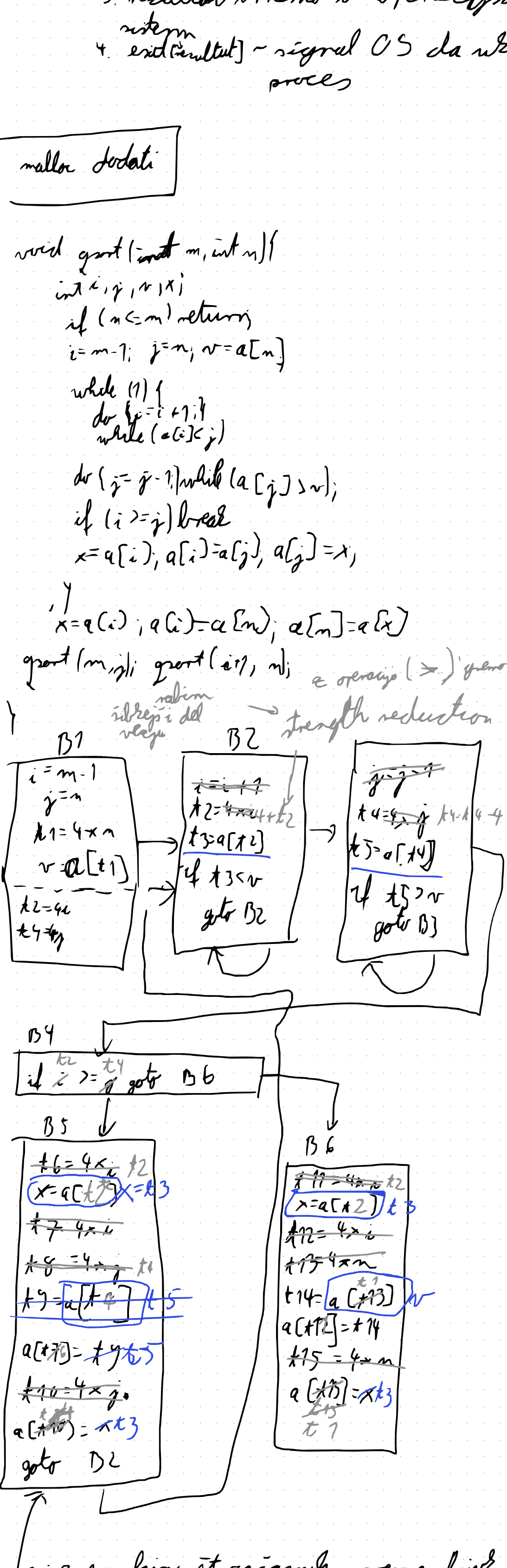
1. pripravi sklad
   - pripravi prazno kopico
   - statične spremenljivke inits jek na 0
2. klic funkcije main
3. rezultat vrnemo v operacijski sistem
4. exit(rezultat) ~ signal OS da ukine proces

malloc dodati

```
void qsort(int m, int n){
  int i, j, v, x;
  if (n<=m) return;
  i=m-1; j=n; v=a[n];
  while (1) {
    do {i=i+1;}
    while (a[i]<v);
    do {j=j-1;} while (a[j]>v);
    if (i>=j) break;
    x=a[i]; a[i]=a[j]; a[j]=x;
  }
  x=a[i]; a[i]=a[n]; a[n]=a[x];
  qsort(m,j); qsort(i+1,n);
}
```

a operacija (>) ima manj ALU

rabim izhreje i del vsega → strength reduction



ni zmer fajn št začasnih spremenljivk

zvotraj zanke smo ukinili z množenji

gledamo kje se ista podizraza pojavlja in spremenljivke x ne spreminjajo

copy propagation

loop-invariant variables → dve spremenljivki se skrati spreminjata (i, t2), (j, t4)

induction variables → spremenljivki ki se spreminjata sorazmerno prevlik

15. Predavanje

LR sintaksna analiza

antlr je LL1 (gleda samo trenutni token) da ne pr
vsaten produkcijo gre

LR: 1. gre od spodaj navzgor
2. zgradi drevo desno
3. simboli so revers ordered



dobimo a, ε, a, A, S

LR je sposobna analizirati več KNJ

primer
S → a B
S → A c
S → A d
A → a
B → b

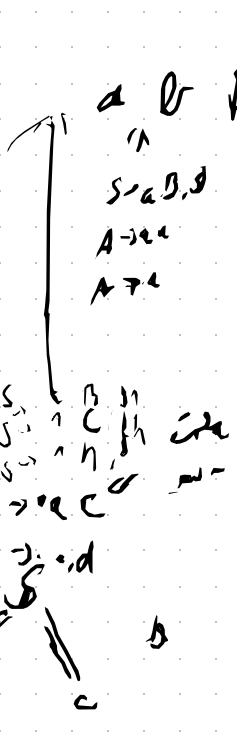
← to ni LL1, ker ko vidimo v bralni 'a' ne vemo kam naprej

S = {ab, ac, ad}



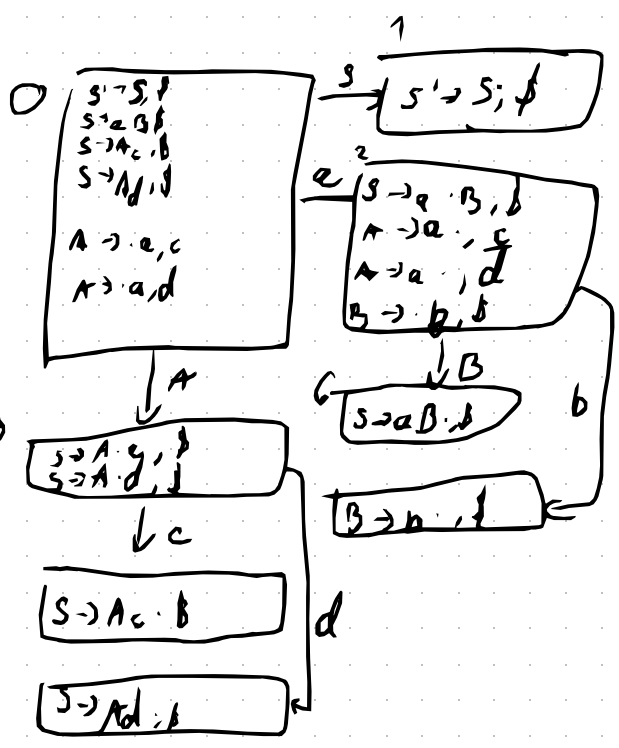
LL ne more izbrati naslednje produkcije

LR: izdere produkcijo za del, ki je že viden





S' → S
S → a B
S → A d
S → A c
A → a
B → b



sklad	vrh	vhod	izhod	
$	0	a c $		shift / premik
$0 a	2	c $		
$0 A	3	c $	A → a	reduce / prevedba
$0 A 3 c	4	$		
$0 S	1	$	S → A C	
$0		a c $	ERROR	
$0 a	2	a $		

LR → da vse prevedbe na sklad, ker se lahko odločiš potem pogledamo vse kar je na skladu

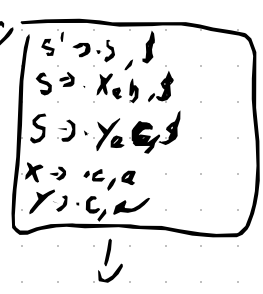
redukcija
I S' → S
II S → a B
III S → A d
IV S → A c
V A → a
VI B → b

	a	b	c	d	$	S	A	B
0	s2					1	3	
1					acc			
2			s7	rV	rV			6
3			s5	s4	rV			
4					rIII			
5					rIV			
6					rII			
7					rVI			

↑ action del LR tabele

↑ goto del LR tabele

S → X a b
S → Y a c
X → c
Y → c

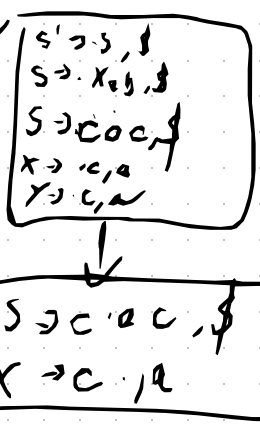


reduce / reduce conflict

napačno med 2 prevedbama

popravljeno

S → X a b
S → c a b
X → c



shift / reduce conf

napačno med premiki in prevedbo